ABSTRACT

Kanivu Cares is a web-based application developed for **Kanivu – A Welfare & Charity Unit**, designed to streamline welfare operations and enhance the delivery of charity services within the community. Built using **HTML, CSS, and JavaScript** for the frontend and **Django with Python** for the backend, the platform delivers a secure, efficient, and user-friendly experience for both internal members and public users.

Serving as a centralized digital hub, **Kanivu Cares** enables Kanivu members to contribute actively by submitting financial reports, maintaining work logs, and engaging in ongoing welfare initiatives. It ensures seamless coordination and transparency among all stakeholders involved in charity projects.

Key Features of Kanivu Cares:

- **Public Users** — Can view current programs, request assistance, or express interest in participating or donating.
- **Kanivu Members Section** — Members can submit financial accounts, maintain daily work logs, and participate in ongoing activities and service requests.
- **Co-ordinator Dashboard** — With limited access rights compared to convenors, assistant convenors help manage member activities and logs under the supervision of the convenors. They serve as support leads to ensure daily operations proceed smoothly.
- **Convenor Dashboard** — Users with convenor roles are granted elevated privileges to oversee member contributions, review and approve work logs, manage events, and facilitate welfare initiatives. Convenors play a key role in operational coordination and volunteer oversight.
- **Administrator Panel (Admin)** — Provides full system control, including user role management, platform maintenance, and governance. The admin ensures secure and reliable operations across the platform.

By integrating internal functions—such as volunteer tracking and financial reporting—with public-facing capabilities like aid requests and program visibility, **Kanivu Cares** fosters transparency and encourages community engagement. The platform promotes collaboration at every level, reinforcing Kanivu's foundational values of **compassion, accountability, and service**.

More than just a management system, **Kanivu Cares** represents a vital step in digitizing grassroots charity work—empowering volunteers, convenors, administrators, and the broader public to engage in impactful social service.

CHAPTER 1 INTRODUCTION

1.1 ABOUT THE PROJECT

Kanivu Cares is the official digital platform of **Kanivu – A Welfare & Charity Unit**, dedicated to advancing our mission of compassion, service, and community welfare. Designed to support and streamline charitable initiatives, the platform enables seamless coordination, transparent operations, and active engagement across all levels of our organization.

Built using **HTML, CSS, JavaScript, Django, and Python**, **Kanivu Cares** empowers Kanivu members to request support for distributing essential kits, organize welfare and charity programs, maintain financial records, and document daily volunteer activities. It provides a structured environment where collaboration thrives—featuring dedicated access for **Convenors**, **Assistant Convenors**, **Active Members**, and the **Admin Team**.

With a secure, user-friendly interface, the platform serves both internal stakeholders and the public. Members can manage their contributions and initiatives, while public users can request aid, view current projects, or express their interest in volunteering or donating.

By digitizing grassroots welfare work, **Kanivu Cares** bridges the gap between service and technology—fostering accountability, enhancing visibility, and strengthening the spirit of giving within the community.

MODULE DESCRIPTION

The Kanivu Cares platform is divided into several key modules, each tailored to manage specific responsibilities within the welfare and charity ecosystem. Below is a brief overview of each module and its core functionality:

- PUBLIC USER MODULE
- MEMBER MODULE
- CO-ORDINATOR MODULE
- CONVENOR MODULE
- ADMINISTRATOR MODULE

1.1.1 PUBLIC USER MODULE

To inform and engage the general public, offering transparency about ongoing activities and enabling outreach and assistance.

Key Features:

- **View Welfare Programs:** Access detailed information about ongoing and past charity events.
- **Contact Kanivu:** Simplified contact options to get in touch with members or coordinators
- **Announcements & News:** Display recent activities, success stories, and upcoming events.

1.1.2 MEMBER MODULE

To provide Kanivu members with tools to participate actively in charity and welfare initiatives while maintaining transparency and accountability.

Key Features:

- **View-Only Access:** Members have restricted permissions to **view the member list** and **financial reports** for transparency, without the ability to modify sensitive data.
- **Duty Roles Allocation:** Members will be assigned to teams such as **Collection**, **Finance**, **Event**, or **Team Controller** based on operational needs. *(Note: This feature is planned for a future major update and is not currently active.)*
- **Submit Accounts:** Members can upload expense details and financial records related to charity activities.
- **Notifications & Updates:** Receive alerts and updates related to approvals, upcoming programs, or messages from coordinators/admins.

1.1.3 CO-ORDINATOR MODULE

To assist convenor in managing daily member activities with limited administrative privileges, acting as a bridge between members and convenor.

Key Features:

- **Member Account Creation & Management:** Can create new member accounts which are submitted as requests to the Convenor for approval. All pending requests can be tracked. If rejected, coordinators can review, update, and resubmit the application. Additionally, coordinators have the ability to remove (kick) existing members when necessary.
- **Internal Notifications & Announcements:** Can able to generate, edit, or delete announcements for collections, events, meetings, programs, or any internal updates, keeping all members informed and engaged.
- **Financial Record Management:** Can create, edit, or delete financial entries related to collections, events, income/expenses, etc. If an incorrect or suspicious entry is identified, coordinators can flag and notify for review or correction.
- **Aid Kit Receiver Management:** Able to add, edit, or delete beneficiary information including photo, address, and geolocation. Can also notify issues or inconsistencies in receiver details for necessary action.

1.1.4 CO-ORDINATOR MODULE

To allow Kanivu coordinators to manage and validate member contributions, oversee event planning, and ensure smooth execution of welfare tasks

Key Features:

- **Full Member Account Control:** Convenors have complete authority to create and approve new member accounts. They can also review and approve/reject account requests submitted by Coordinators. If declined due to incorrect or incomplete entries, the request can be sent back for revision and resubmission.

- **Full Member Account Control:** Can assign, update, or change a member's duty role (e.g., Collection, Finance, Events, or Team Controller) at any time, either during account creation or later.
- **Full Member Account Control:** Can create and send notifications to members regarding collections, meetings, upcoming programs, events, or general updates, helping maintain active communication within the unit.
- **Full Member Account Control:** Can create, edit, or delete financial records involving collection reports, event-related income/expenses, and other welfare activities. They can also verify and finalize records submitted by Coordinators.
- **Full Member Account Control:** Has access to add, edit, or delete the personal details of aid kit beneficiaries, including their photo, address, and live location using map-based tracking for verification and documentation purposes.

1.1.5 ADMIN MODULE

To oversee the overall functionality, integrity, and security of the platform. Admins ensure smooth operation and governance across all sections.

Key Features:

- **User Management:** Add, remove, or modify member, coordinator, and public user access.
- **Content Control:** Update news, announcements, and site-wide content.
- **Access Permissions:** Define and control what each user role can view or modify.
- **System Monitoring:** Monitor platform activity and logs to ensure compliance and data integrity.
- **Backup & Maintenance Tools:** Admin tools for backups, updates, and overall system health.

CHAPTER 2
SYSTEM ANALYSIS

System analysis works with users to identify goals and build systems to achieve them. System analysis is the important phase of any system development process. The system is studied to the minutest details and analyzed. The system analyst plays the role of an interrogator and dwells deep into the working of the present system. In analysis, a detailed study of these operations performed by a system and their relationship within and outside of the system is done. A key question considered here is "what must be done to solve the problem?" The system is viewed as a whole and the input to the system are identified. The output from the system are traced through the various data are collected on available files, decision points and transaction handled by present system. After all, this proposal becomes a formal agreement that paves way for the design implementation.

2.1 REQUIREMENT ANALYSIS

Requirement analysis is the process of analyzing the requirements with the potential goal of improving or modifying the existing system and its features. Analyzing the advantages and disadvantages of the current system, the proposed system can be designed which can avoid all the complexities, inabilities and the disadvantages of the existing system. The new system requirements are defined during this phase. The requirements of the desired software product are extracted.

To design a new system we need the requirements of the system. Based on the business scenario the software Requirement Specification document is prepared in this phase. The specifications are intended to guide the group through the development process. It explains all the process, activities, relationships and all other organizational objectives. With the analysis of the current system the analyst can understand the reason for automation and what effects the automation system might have.

User's requirements have been identified as follows:

- **Pre-defined Questions**

It allows analysts to collect information about the various aspects of the system from large number of persons. The use of standardized question format can yield more reliable data than other technique.

- **Interview**

Analysts use interviews to collect information from individuals or from groups. The respondents are generally current users of the existing system.

- **Record Interview**

In record review, analysts examine information that has been recorded about the system and the users. Record inspection can be performed at the beginning of the study as an introduction and renew later in the study as a basis for comparing actual operations with what the records indicate.

PROJECT PLANNING

Even with small projects you need to know what the project aims to do, how you'll know if you've finished (and been successful, what the scope of the project will be, what resources it will be able to use, and who will be involved, but that's just the beginning.

Start a mind map if you haven't already. Think about what the project aims to produce, what it covers, what you will need to finish it and then draft the first ideas for a timetable:

Scope may be the trickiest. It should mention what's included, but it's a good idea to try to write what is **not included** as well. No, not trivial exclusions ("this project will exclude an attempt to set up a moon-base"). What's hard is to catch as many as you can of the exclusions that other player in the project might expect to be **included**.

You may not have decided much of what you write about aims, scope and resources yourself -- maybe your boss told you -- but whether you define them and then go seeking agreement, or they are handed down, make sure they have been thought through, written down and circulated. Either way if you are managing the project, the next part is down to you. And this is where the mindmap gets really useful.

Mind mapping encourages structured capture of unstructured thoughts. You could do some of this with a checklist, but a mind map lets you move quickly, skip from one area to another when you foresee consequences there, and remain open and unconstrained when planning. There will be a time for formal and detailed planning, but this is not it -- you need a broad view and the mind map approach gives that.

Next, develop the mind map to a more detailed level. You have aims, now you need a statement of requirements, and an informal agreement with the sponsors of the project, or more likely a formal agreement, and even a legal contract in some cases. Depending on how formal your sponsors are, there may be penalties to agree at this stage and you will need to consider contingencies in cases of failure to obtain all the resources or meet certain requirements -- delivery dates especially.

The problem with linear planning is clear, now. Here the description, as is the nature of prose, is linear, but at this point in the project planning, there will actually be movement on many fronts -- timetable planning; consideration of constraints; negotiation about completion dates and penalties; factoring in the need for approvals and whether these will be available in a timely manner; and considering internal controls imposed by, say, quality processes or regulatory requirements, if these apply.

Such multiple and parallel activities are better supported by a mind map than a linear outline, a checklist or a purely text-based planning document.

This mind map continues to grow though the planning process as more and more detail is filled in, and detailed budgeting, formal timetable preparations and resource allocation get under way.

Taking regular snapshots of the mind map and keeping these intermediate stages is a good idea. It can help with future projects. Once you have found the right form of project planning mind map for your work, you will be able to use its general structure for project after project, and tune it as your projects grow.

2.2 EXISTING SYSTEM

Currently, there is no official or structured digital system in place for the management of KANIVU, the Charity & Welfare Unit of CHMM College.

- Current Practices:

➢ All activities, communications, and updates are handled manually or through informal channels such as:
  - WhatsApp groups
  - Physical notices
  - Verbal coordination among members

➢ Financial records are maintained in handwritten ledgers or offline spreadsheets.
➢ Member details and roles are tracked manually.
➢ Public interaction or help requests are not formally documented, leading to poor tracking.

- Challenges of the Current (Non-Systematic) Approach:

➢ Lack of transparency and accountability
➢ Risk of data loss and miscommunication
➢ No official platform to represent KANIVU online
➢ Difficult to manage member details and financial records
➢ No way for the public to directly reach out for help or stay updated on activities

2.3 PROPOSED SYSTEM

- SYSTEM TITLE:

KANIVU CARES – A Charity & Welfare Management Website for KANIVU, CHMM College.

- SYSTEM OBJECTIVE:

To design and implement a centralized online platform that streamlines the activities, member management, financial transparency, and communication of KANIVU – the Charity & Welfare Unit of CHMM College.

This website serves as the first digital initiative by KANIVU to reach both internal stakeholders and the general public, ensuring structured and effective delivery of charitable services.

It is an analysis of the viability of an idea. It is the second major step in the system development life cycle. It focuses on helping answer the essential question of “should we proceed with the proposed project idea?”

The feasibility study is an evaluation and analysis of the potential of a proposed project which is based on extensive investigation and research to support the process of decision making. A feasibility study evaluates the project's potential for success.

Different feasibility tests are :

1. Economic Feasibility
2. Technical Feasibility
3. Operational Feasibility

ECONOMIC FEASIBILITY

Economic analysis is also known as cost/benefit analysis. It is the most frequently used method for evaluating the effectiveness of a new system. In economic analysis, the procedure is to determine the benefits and savings that are expected from a candidate system and compare them with costs.

The proposed system is economically feasible due to the following reasons :

- Minimal if built in-house or by students/developers using open-source tools.
- Cloud hosting and domain registration are low-cost (approx. ₹3000–₹5000/year).
- Overall cost is moderate, includes hosting, occasional bug fixes, and data backup.

TECHNICAL FEASIBILITY

It considers the technical requirements of the proposed project. The technical requirements are then compared to the technical capability of the organization. The systems project is considered technically feasible if the internal technical capability is sufficient to support the project requirements.

Our proposed system is technically feasible for the following reasons :

- All required technologies are mature, open-source (or freemium), and welldocumented.
- Developers can easily implement CRUD operations, search filters, authentication, and admin dashboards with available frameworks.
- Scalable architecture possible if the user base grows.

OPERATIONAL FEASIBILITY

Proposed project would be beneficial only if they can be turned into information system that'll meet the organization operating requirements. Once the main problems faced during the development of a new system is getting acceptance from user. The test of operational feasibility asks if the system will work when developed and installed.

Our proposed system is operationally feasible due to the following reasons:

- The tools selected for developing the software are PHP, MySQL, Windows 10 or higher which are all easily available.
- It can reduce different difficulties in the existing system.

- User Perspective:
  - Simple and intuitive interface encourages participation from students, staff, and the general public.
  - Reduces dependency on physical notice boards or word-of-mouth communication.
- Admin Perspective:
  - Streamlined management of item reports.
  - Easy to moderate and generate reports.
- Challenges:

- Need to ensure genuine reporting and prevent misuse.
- User awareness campaigns might be required initially.

LEGAL AND ETHICAL FEASIBILITY

Proposed project would be beneficial only if they can be turned into information system that'll meet the organization operating requirements. Once the main problems faced during the development of a new system is getting acceptance from user. The test of operational feasibility asks if the system will work when developed and installed.

Our proposed system is operationally feasible due to the following reasons:

- Privacy: User data such as contact info must be protected per IT Act 2000 and related data protection norms.
- Content Moderation: Admin must ensure no offensive or misleading content is posted.
- Terms of Use: Clear disclaimers needed stating that the platform helps connect people but doesn't guarantee item recovery.

SCHEDULE FEASIBILITY

Schedule feasibility assesses whether a proposed system can be developed and implemented within the available time frame. It takes into account the complexity of the system, resource availability, and the skills of the development team. A well-defined timeline, broken into manageable phases such as planning, design, development, testing, and deployment, helps ensure timely delivery. If the estimated schedule aligns with the project goals and does not exceed critical deadlines, the system is considered schedulefeasible.

Our proposed system is legally and ethically feasible due to the following reasons:

- Estimated Timeline (for MVP):
- Requirement Analysis – 1 week
- UI/UX Design – 1 week
- Backend & Database – 2 weeks
- Frontend Integration – 2 weeks
- Testing & Deployment – 1 week
- Total: 7 Weeks

2.4 SYSTEM SPECIFICATION

Hardware Requirements:

Hardware	Minimum	Recommended
Processor	Intel i3/Ryzen 3/Apple M1	Intel i5, Ryzen 5, Apple M1, M2
Ram	4 GB	8 GB

Storage	128 GB HDD	256 SSD
Display	1366x768	External (optional for multitask)
Graphics	Integrated	Integrated/Dedicated

Software Requirements:

Operating System:

• Windows 10 (64-bit) or later
• macOS Monterey or later
• Linux (Ubuntu 20.04 or newer)

Development Stack:

• Python 3.10+
• Django 4.x
• SQLite (for dev) / PostgreSQL (for production)
• Node.js v20+ (for frontend tools like Webpack, Tailwind CSS if used)
• Visual Studio Code or any preferred IDE
• Git for version control
• Browser (Chrome, Firefox, Edge)

2.5.1 SOFTWARE REQUIREMENTS

In this section a brief introduction of the technologies of the software used in the proposed system are discussed

Visual Studio 2022

Microsoft's Visual Studio is one of the most popular developer IDEs in the world. Not only is Visual Studio modern, feature-rich, and advanced, but it also gets more frequent updates than any other IDE out there. The next version of Visual Studio is going to be Visual Studio 2022. The Visual Studio team just released the first public preview of Visual Studio 2022.

Visual Studio 2022 is much faster and more lightweight compared to the current version. The Visual Studio team has focused on making IDE more user-friendly by enhancing the overall user experience. It looks cleaner.

Visual Studio 2022 also has intelligent features that include recommendations around code cleanness, quality, standards, and best practices

Visual Studio 2022

Removed the ability to revert back to the Team Explorer Git UI, making the new Git experience the only available built-in tooling.

Removed the option to install the GitHub extension from the Visual Studio Installer. New versions of the test platform will not be able to run Generic tests and Ordered tests.

Web Tools

The Publish summary page now has actions to start/stop remote debugging and profiling under the '...' menu on the top right corner of the 'Hosting' section

The Connected Services page now has an action to launch Storage Explorer

The "ASP.NET Core Empty" template that comes with .NET 6 is using the new 'minimal

APIs' paradigm for which we have started to add support.

Some of the feature that has not implemented yet but is coming in the next releases.

- Web Live Preview
- Instrumentation profiler
- Azure Cloud Service project support
- T-SQL debugger
- Web Load Test and TestController/TestAgent
- Azure DataLake
- Coded UI Test
- DotFuscator
- Incredibuild IDE integration
- IntelliCode find and replace by example

Visual Studio is an integrated development environment (IDE) developed by Microsoft. It is used to develop computer programs including websites, web apps, web services and mobile apps. Visual Studio uses Microsoft software development platforms such as Windows API, Windows Forms, Windows Presentation Foundation (WPF), Windows Store and Microsoft Silverlight. It can produce both native code and managed code.

Visual Studio includes a code editor supporting IntelliSense (the code completion component) as well as code refactoring. The integrated debugger works as both a source-level debugger and as a machine-level debugger. Other built-in tools include a code profiler, designer for building GUI applications, web designer, class designer, and database schema designer. It accepts plug-ins that expand the functionality at almost every level including adding support for source control systems (like Subversion and Git) and adding new toolsets like editors and visual designers for domain-specific languages or toolsets for other aspects of the software development lifecycle (like the Azure DevOps client: Team Explorer).

Visual Studio supports 36 different programming languages and allows the code editor and debugger to support (to varying degrees) nearly any programming language, provided a language-specific service exists. Built-in languages include C, C++, C++/CLI, Visual Basic .NET, C#, F#, JavaScript, TypeScript, XML, XSLT, HTML, and CSS. Support for other languages such as Python, Ruby, Node.js, and M among others is available via plugins. Java (and J#) were supported in the past.

Architecture

Visual Studio does not support any programming language, solution or tool intrinsically; instead, it allows the plugging of functionality coded as a VSPackage. When installed, the functionality is available as a Service. The IDE provides three services: SVsSolution, which provides the ability to enumerate projects and solutions; SVsUIShell, which provides windowing and UI functionality (including tabs, toolbars, and tool windows); and SVsShell, which deals with registration of VSPackages. In addition, the IDE is also responsible for coordinating and enabling communication between services.[10] All editors, designers, project types and other tools are implemented as VSPackages. Visual Studio uses COM to access the VSPackages. The Visual Studio SDK also includes the Managed Package Framework (MPF), which is a set of managed wrappers around the COMinterfaces that allow the Packages to be written in any CLI compliant language.[11] However, MPF does not provide all the functionality exposed by the Visual Studio COM interfaces.[12] The services can then be consumed for creation of other packages, which add functionality to the Visual Studio IDE.

Support for programming languages is added by using a specific VSPackage called a Language Service. A language service defines various interfaces which the VSPackage implementation can implement to add support for various functionalities. Functionalities

that can be added this way include syntax coloring, statement completion, brace matching, parameter information tooltips, member lists, and error markers for background compilation. If the interface is implemented, the functionality will be available for the language. Language services are implemented on a per-language basis. The implementations can reuse code from the parser or the compiler for the language. Language services can be implemented either in native code or managed code. For native code, either the native COM interfaces or the Babel Framework (part of Visual Studio SDK) can be used. For managed code, the MPF includes wrappers for writing managed language services.

Visual Studio does not include any source control support built in but it defines two alternative ways for source control systems to integrate with the IDE. A Source Control VSPackage can provide its own customised user interface. In contrast, a source control plugin using the MSSCCI (Microsoft Source Code Control Interface) provides a set of functions that are used to implement various source control functionality, with a standard Visual Studio user interface. MSSCCI was first used to integrate Visual SourceSafe with Visual Studio 6.0 but was later opened up via the Visual Studio SDK. Visual Studio .NET 2002 used MSSCCI 1.1, and Visual Studio .NET 2003 used MSSCCI 1.2. Visual Studio 2005, 2008, and 2010 use MSSCCI Version 1.3, which adds support for rename and delete propagation, as well as asynchronous opening.

Visual Studio supports running multiple instances of the environment (each with its own set of VSPackages). The instances use different registry hives (see MSDN's definition of the term "registry **hive**" in the sense used here) to store their configuration state and are differentiated by their AppId (Application ID). The instances are launched by an AppIdspecific .exe that selects the AppId, sets the root hive, and launches the IDE. VSPackages registered for one AppId are integrated with other VSPackages for that AppId. The various product editions of Visual Studio are created using the different AppIds. The Visual Studio Express edition products are installed with their own AppIds, but the Standard, Professional, and Team Suite products share the same AppId. Consequently, one can install the Express editions side-by-side with other editions, unlike the other editions which update the same installation. The professional edition includes a superset of the

VSPackages in the standard edition, and the team suite includes a superset of the VSPackages in both other editions. The AppId system is leveraged by the Visual Studio Shell in Visual Studio 2008.

Features Code Editor

Visual Studio includes a code editor that supports syntax highlighting and code completion using IntelliSense for variables, functions, methods, loops, and LINQ queries. IntelliSense is supported for the included languages, as well as for XML, Cascading Style Sheets, and JavaScript when developing web sites and web applications. Autocomplete suggestions appear in a modeless list box over the code editor window, in proximity of the editing cursor. In Visual Studio 2008 onwards, it can be made temporarily semi-transparent to see the code obstructed by it. The code editor is used for all supported languages.

The code editor in Visual Studio also supports setting bookmarks in code for quick navigation. Other navigational aids include collapsing code blocks and incremental search, in addition to normal text search and regex search. The code editor also includes a multiitem clipboard and a task list. The code editor supports code snippets, which are saved templates for repetitive code and can be inserted into code and customized for the project being worked on. A management tool for code snippets is built in as well. These tools are surfaced as floating windows which can be set to automatically hide when unused or docked to the side of the screen. The code editor in Visual Studio also supports code refactoring including parameter reordering, variable and method renaming, interface extraction, and encapsulation of class members inside properties, among others.

Debugger

Visual Studio includes a debugger that works both as a source-level debugger and as a machine-level debugger. It works with both managed code as well as native code and can be used for debugging applications written in any language supported by Visual Studio. In addition, it can also attach to running processes, monitor, and debug those processes.[24] If source code for the running process is available, it displays the code as it is being run. If source code is not available, it can show the disassembly. The Visual Studio debugger can also create memory dumps as well as load them later for debugging. Multi-threaded programs are also supported. The debugger can be configured to be launched when an application running outside the Visual Studio environment crashes.

The Visual Studio Debugger allows setting breakpoints (which allow execution to be stopped temporarily at a certain position) and watches (which monitor the values of variables as the execution progresses). Breakpoints can be conditional, meaning they get triggered when the condition is met. Code can be stepped over, i.e., run one line (of source code) at a time. It can either step into functions to debug inside it, or step over it, i.e., the execution of the function body isn't available for manual inspection. The debugger supports Edit and Continue, i.e., it allows code to be edited as it is being debugged. When debugging, if the mouse pointer hovers over any variable, its current value is displayed in a tooltip ("data tooltips"), where it can also be modified if desired. During coding, the Visual Studio debugger lets certain functions be invoked manually from the Immediate tool window. The parameters to the method are supplied at the Immediate window.

Designer[edit]

Visual Studio includes many visual designers to aid in the development of applications.

These tools include:

Windows Forms Designer

The Windows Forms designer is used to build GUI applications using Windows Forms. Layout can be controlled by housing the controls inside other containers or locking them to the side of the form. Controls that display data (like textbox, list box and grid view) can be bound to data sources like databases or queries. Data-bound controls can be created by dragging items from the Data Sources window onto a design surface.[29] The UI is linked with code using an event-driven programming model. The designer generates either C# or VB.NET code for the application.

WPF Designer

The WPF designer, codenamed Cider, was introduced with Visual Studio 2008. Like the Windows Forms designer it supports the drag and drop metaphor. It is used to author user interfaces targeting Windows Presentation Foundation. It supports all WPF functionality including data binding and automatic layout management. It generates XAML code for the UI. The generated XAML file is compatible with Microsoft Expression Design, the designer-oriented product. The XAML code is linked with code using a codehttps://en.wikipedia.org/wiki/Code-behindbehind model.

Web designer/development

Visual Studio also includes a web-site editor and designer that allows web pages to be authored by dragging and dropping widgets. It is used for developing ASP.NET applications and supports HTML, CSS and JavaScript. It uses a code-behind model to link with ASP.NET code. From Visual Studio 2008 onwards, the layout engine used by the web designer is shared with the discontinued Expression Web. There is also ASP.NET MVC support for MVC technology as a separate

download and ASP.NET Dynamic Data project available from Microsoft.[32]

Class designer

The Class Designer is used to author and edit the classes (including its members and their access) using UML modeling. The Class Designer can generate C# and VB.NET code outlines for the classes and methods. It can also generate class diagrams from hand-written classes.

Data designer

The data designer can be used to graphically edit database schemas, including typed tables, primary and foreign keys and constraints. It can also be used to design queries from the graphical view.

Mapping designer

From Visual Studio 2008 onwards, the mapping designer is used by LINQ to SQL to design the mapping between database schemas and the classes that encapsulate the data. The new solution from ORM approach, ADO.NET Entity Framework, replaces and improves the old technology. **Other tools**

Properties Editor

The Properties Editor tool is used to edit properties in a GUI pane inside Visual Studio. It lists all available properties (both read-only and those which can be set) for all objects including classes, forms, web pages and other items.

Object Browser

The Object Browser is a namespace and class library browser for Microsoft .NET. It can be used to browse the namespaces (which are arranged hierarchically) in managed assemblies. The hierarchy may or may not reflect the organization in the file system.

Solution Explorer

In Visual Studio parlance, a solution is a set of code files and other resources that are used to build an application. The files in a solution are arranged hierarchically, which might or might not reflect the organization in the file system. The Solution Explorer is used to manage and browse the files in a solution.

Team Explorer

Team Explorer is used to integrate the capabilities of Azure DevOps (either Azure DevOps Services or Azure DevOps Server) into the IDE . In addition to version control integration it provides the ability to view and manage individual work items (including user stories, bugs, tasks and other documents). It is included as part of a Visual Studio installation and is also available as a standalone download.[33][34]

Data Explorer

Data Explorer is used to manage databases on Microsoft SQL Server instances. It allows creation and alteration of database tables (either by issuing T-SQL commands or by using the Data designer). It can also be used to create queries and stored procedures, with the latter in either T-SQL or in managed code via SQL CLR. Debugging and IntelliSense support is available as well.

Server Explorer

The Server Explorer tool is used to manage database connections on an accessible computer. It is also used to browse running Windows Services, performance counters, Windows Event Log and message queues and use them as a datasource.

Dotfuscator Community Edition

Visual Studio includes a free 'light' version of Dotfuscator

Text Generation Framework

Visual Studio includes a full text generation framework called T4 which enables Visual Studio to generate text files from templates either in the IDE or via code.

ASP.NET Web Site Administration Tool

The ASP.NET Web Site Administration Tool allows for the configuration of ASP.NET websites. **Visual Studio Tools for Office**

Visual Studio Tools for Office is a SDK and an add-in for Visual Studio that includes tools for developing for the Microsoft Office suite. Previously (for Visual Studio .NET 2003 and Visual Studio 2005) it was a separate SKU that supported only Visual C# and Visual Basic languages or was included in the Team Suite. With Visual Studio 2008, it is no longer a separate SKU but is included with Professional and higher editions. A separate runtime is required when deploying VSTO solutions.

Testing tools

Microsoft Visual Studio can write high-quality code with comprehensive testing tools to aid in the development of applications. These tools include:

Unit testing, IntelliTest, Live Unit Testing, Test Explorer, CodeLens test indicators, Code coverage analysis, Fakes

ASP.Net

ASP.NET is an open source web framework, created by Microsoft, for building modern web apps and services with .NET. ASP.NET is cross platform and runs on Windows, Linux, macOS, and Docker. ASP.NET is a web application framework designed and developed by Microsoft. ASP.NET is open source and a subset of the **.NET Framework** and successor of the classic ASP(*A*ctive *S*erver *P*ages). With version 1.0 of the .NET Framework, it was first released in January 2002. So a question comes to mind that which technology we were using before the year 2002 for developing web applications and services? Answer is *Classic ASP*. So before .NET and ASP.NET there was Classic ASP. ASP.NET is built on the **CLR(Common Language Runtime)** which allows the programmers to execute its code using any .NET language(C#, VB etc.). It is specially designed to work with HTTP and for web developers to create dynamic web pages, web applications, web sites, and web services as it provides a good integration of HTML, CSS, and JavaScript. .NET Framework is used to create a variety of applications and services like Console, Web, and Windows, etc. But ASP.NET is only used to create web applications and web services. A web application is an application installed only on the web server which is accessed by the users using a web browser like Microsoft Internet Explorer, Google Chrome, Mozilla FireFox, Apple Safari, etc. There are also some other technology like Java, PHP, Perl, Ruby on Rails, etc. which can be used to develop web applications. Web applications provide the cross-platform feature. The user needs only a web browser to access a web application. The web applications which are developed using the .NET framework or its subsets required to execute under the *Microsoft Internet Information Services(IIS)* on the server side. Don't confuse in the terms ASP.NET, ASP.NET core, ASP.NET MVC, etc. ASP(Active Server Pages) supports a lot of development models which are as follows:

- Classic ASP: It is the first server side scripting language developed by Microsoft.
- ASP.NET: It is web development framework and successor of Classic ASP. ASP.NET 4.6 is the latest version.

- ASP.NET Core: In November 2015, Microsoft released the 5.0 version of ASP.NET which get separated later and known as ASP.NET Core. Also, it is considered as an important redesign of ASP.NET with the feature of open-source and cross-platform. Before this version, ASP.NET is only considered as Windows-only version.
- ASP.NET Web Forms: These are the event-driven application model which are not considered a part of the new ASP.NET Core. These are used to provide the serverside events and controls to develop a web application.
- ASP.NET MVC: It is the Model-View-Controller application model which can be merged with the new ASP.NET Core. It is used to build dynamic websites as it provides fast development.
- ASP.NET Web Pages: These are the single page application which can be merged into ASP.NET Core.
- ASP.NET API: It is the Web Application Programming Interface(API).

Also, to create web applications ASP.NET provide the 3 development styles which are ASP.NET Web Pages, ASP.NET MVC, Web Forms.

There are a lot of reasons which makes the ASP.NET popular among the developers. Some of the reasons are listed below: Extending .NET Framework: ASP.NET is a subset of .NET Framework as it extends the .NET Framework with some libraries and tools to develop web apps. The thing that it adds to the .NET Framework is Libraries for common web patterns like MVC, Editor Extensions, the base framework to process the web requests, and web-page templating syntax like Razor, etc. Performance: It is faster than the other web frameworks available in the market. Backend Code: With the help of ASP.NET you can write the backend code for data access and any logic in C#. Dynamic Pages: In ASP.NET,

Razor provides the syntax for developing the dynamic web pages with the help of C# and HTML. ASP.NET can be integrated with JS(JavaScript) and it also includes the frameworks like React and Angular for the SPA(Single Page Application.) Supporting different OS: You can develop and execute ASP.NET apps on

Windows, Linux, Docker, and MacOS. The Visual Studio provides the tools to build .NET apps different OS.

ASP.NET Web Forms Model

ASP.NET web forms extend the event-driven model of interaction to the web applications. The browser submits a web form to the web server and the server returns a full markup page or HTML page in response.

All client side user activities are forwarded to the server for stateful processing. The server processes the output of the client actions and triggers the reactions.

Now, HTTP is a stateless protocol. ASP.NET framework helps in storing the information regarding the state of the application, which consists of:

- Page state
- Session state

The page state is the state of the client, i.e., the content of various input fields in the web form. The session state is the collective information obtained from various pages the user visited and worked with, i.e., the overall session state. To clear the concept, let us take an example of a shopping cart.

User adds items to a shopping cart. Items are selected from a page, say the items page, and the total collected items and price are shown on a different page, say the cart page. Only HTTP cannot keep track of all the information coming from various pages. ASP.NET session state and server side infrastructure keeps track of the information collected globally over a session.

The ASP.NET runtime carries the page state to and from the server across page requests while generating ASP.NET runtime codes, and incorporates the state of the server side components in hidden fields.

This way, the server becomes aware of the overall application state and operates in a twotiered connected way.

The ASP.NET Component Model

The ASP.NET component model provides various building blocks of ASP.NET pages. Basically it is an object model, which describes:

- Server side counterparts of almost all HTML elements or tags, such as <form> and <input>.
- Server controls, which help in developing complex user-interface. For example, the Calendar control or the Gridview control.

ASP.NET is a technology, which works on the .Net framework that contains all webrelated functionalities. The .Net framework is made of an object-oriented hierarchy. An ASP.NET web application is made of pages. When a user requests an ASP.NET page, the IIS delegates the processing of the page to the ASP.NET runtime system.

The ASP.NET runtime transforms the .aspx page into an instance of a class, which inherits from the base class page of the .Net framework. Therefore, each ASP.NET page is an object and all its components i.e., the server-side controls are also objects.

Components of .Net Framework 3.5

Before going to the next session on Visual Studio.Net, let us go through at the various components of the .Net framework 3.5. The following table describes the components of the .Net framework 3.5 and the job they perform:

Components and their Description

(1) Common Language Runtime or CLR

It performs memory management, exception handling, debugging, security checking, thread execution, code execution, code safety, verification, and compilation. The code that is directly managed by the CLR is called the managed code. When the managed code is compiled, the compiler converts the source code into a CPU independent intermediate language (IL) code.

A Just In Time(JIT) compiler compiles the IL code into native code, which is CPU specific.

(2) .Net Framework Class Library

It contains a huge library of reusable types. classes, interfaces, structures, and enumerated values, which are collectively called types.

(3) Common Language Specification

It contains the specifications for the .Net supported languages and implementation of language integration.

(4) Common Type System

It provides guidelines for declaring, using, and managing types at runtime, and cross-language communication.

(5) Metadata and Assemblies

Metadata is the binary information describing the program, which is either stored in a portable executable file (PE) or in the memory. Assembly is a logical unit consisting of the assembly manifest, type metadata, IL code, and a set of resources like image files.

(6) Windows Forms

Windows Forms contain the graphical representation of any window displayed in the application.

(7) ASP.NET and ASP.NET AJAX

ASP.NET is the web development model and AJAX is an extension of ASP.NET for developing and implementing AJAX functionality. ASP.NET AJAX contains the components that allow the developer to update data on a website without a complete reload of the page.

(8) ADO.NET

It is the technology used for working with data and databases. It provides access to data sources like SQL server, OLE DB, XML etc. The ADO.NET allows connection to data sources for retrieving, manipulating, and updating data.

(9) Windows Workflow Foundation (WF)

It helps in building workflow-based applications in Windows. It contains activities, workflow runtime, workflow designer, and a rules engine.

(10) Windows Presentation Foundation

It provides a separation between the user interface and the business logic. It helps in developing visually stunning interfaces using documents, media, two and three dimensional graphics, animations, and more.

(11) Windows Communication Foundation (WCF)

It is the technology used for building and executing connected systems.

(12) Windows CardSpace

It provides safety for accessing resources and sharing personal information on the internet.

(13) LINQ

It imparts data querying capabilities to .Net languages using a syntax which is similar to the tradition query language SQL.

ASP.NET provides an abstraction layer on top of HTTP on which the web applications are built. It provides high-level entities such as classes and components within an objectoriented paradigm.

The key development tool for building ASP.NET applications and front ends is Visual Studio. In this tutorial, we work with Visual Studio 2008.

Visual Studio is an integrated development environment for writing, compiling, and debugging the code. It provides a complete set of development tools for building ASP.NET web applications, web services, desktop applications, and mobile applications.

C#.Net

C# is a general-purpose, modern and object-oriented programming language pronounced as **"C sharp"**. It was developed by Microsoft led by Anders Hejlsberg and his team within the .Net initiative and was approved by the European Computer Manufacturers Association (ECMA) and International Standards Organization (ISO). C# is among the languages for Common Language Infrastructure and the current version of C# is version 7.2. C# is a lot similar to Java syntactically and is easy for the users who have knowledge of C, C++ or Java. **A bit about .Net Framework** .Net applications are multi-platform applications and framework can be used from languages like C++, C#, Visual Basic, COBOL etc. It is designed in a manner so that other languages can use it. know more about .Net Framework **Why C#?** C# has many other reasons for being popular and in demand. Few of the reasons are mentioned below:

1. **Easy to start:** C# is a high-level language so it is closer to other popular programming languages like C, C++, and Java and thus becomes easy to learn for anyone.

2. **Widely used for developing Desktop and Web Application:** C# is widely used for developing web applications and Desktop applications. It is one of the most popular languages that is used in professional desktop. If anyone wants to create Microsoft apps, C# is their first choice.
3. **Community:**The larger the community the better it is as new tools and software will be developing to make it better. C# has a large community so the developments are done to make it exist in the system and not become extinct.
4. **Game Development:** C# is widely used in game development and will continue to dominate. C# integrates with Microsoft and thus has a large target audience. The C# features such as Automatic Garbage Collection, interfaces, object-oriented, etc. make C# a popular game developing language.

Features of C#.Net

By design, C# is the programming language that most directly reflects the underlying Common Language Infrastructure (CLI). Most of its intrinsic types correspond to valuetypes implemented by the CLI framework.

However, the language specification does not state the code generation requirements of the compiler: that is, it does not state that a C# compiler must target a Common Language Runtime, or generate Common Intermediate Language (CIL), or generate any other specific format. Theoretically, a C# compiler could generate machine code like traditional compilers of C++ or FORTRAN.

Some notable distinguishing features of C# are:

- There are no global variables or functions. All methods and members must be declared within classes. Static members of public classes can substitute for global variables and functions.
- Local variables cannot shadow variables of the enclosing block, unlike C and C++. Variable shadowing is often considered confusing by C++ texts.
- C# supports a strict Boolean data type, bool. Statements that take conditions, such as while and if, require an expression of a type that implements the true operator, such as the boolean type. While C++ also has a boolean type, it can be freely converted to and from integers, and expressions such as if (a) require only that a is convertible to bool, allowing a to be an int, or a pointer. C# disallows this "integer meaning true or false" approach on the grounds that forcing programmers to use expressions that return exactly bool can prevent certain types of common

programming mistakes in C or C++ such as if (a = b) (use of assignment = instead of equality ==).

- In C#, memory address pointers can only be used within blocks specifically
- marked as unsafe, and programs with unsafe code need appropriate permissions to run. Most object access is done through safe object references, which always either point to a "live" object or have the well-defined null value; it is impossible to obtain a reference to a "dead" object (one which has been garbage collected), or to a random block of memory. An unsafe pointer can point to an instance of a valuetype, array, string, or a block of memory allocated on a stack. Code that is not marked as unsafe can still store and manipulate pointers through the System.IntPtr type, but it cannot dereference them.
- Managed memory cannot be explicitly freed; instead, it is automatically garbage collected. Garbage collection addresses the problem of memory leaks by freeing the programmer of responsibility for releasing memory which is no longer needed.
- In addition to the try...catch construct to handle exceptions, C# has a try...finally construct to guarantee execution of the code in the finally block.
- Multiple inheritances is not supported, although a class can implement any number of interfaces. This was a design decision by the language's lead architect to avoid complication and simplify architectural requirements throughout CLI.
- C# is more type safe than C++. The only implicit conversions by default are those which are considered safe, such as widening of integers. This is enforced at compile-time, during JIT, and, in some cases, at runtime. There are no implicit conversions between booleans and integers, nor between enumeration members and integers (except for literal 0, which can be implicitly converted to any enumerated type). Any user-defined conversion must be explicitly marked as explicit or implicit, unlike C++ copy constructors and conversion operators, which are both implicit by default.
- Enumeration members are placed in their own scope.
- C# provides properties as syntactic sugar for a common pattern in which a pair of methods, accessor (getter) and mutator (setter) encapsulate operations on a single attribute of a class.
- Full type reflection and discovery is available.
- C# currently (as of version 4.0) has 77 reserved words.

- Checked exceptions are not present in C# (in contrast to Java). This has been a conscious decision based on the issues of scalability and versionability.

Microsoft SQL Server 2008

Organizations today are facing several unique data challenges: theproliferation of data and systems across their enterprise; the need to provide employees, customers, and partners with consistent access to that data; the desire to better equipinformation workers with meaningful information to drive informed decisions; and the mandate to control costs without sacrificing application availability, security, or reliability.

The next release of SQL Server is designed to help enterprises address these challenges. SQL Server 2008 is Microsoft's next generation data management and analysis solution that will deliver increased security, scalability, and availability to enterprise data andanalytical applications while making them easier to create, deploy, and manage. Building on the strengths of SQLServer 2000, SQL Server 2008 will provide an integrated data management and analysis solution that will help organizations of any size to:

1. Build and deploy enterprise applications that are more secure, scalable, and reliable.
2. Maximize the productivity of IT by reducing the complexity of creating, deploying, and managing database applications.
3. Empower developers through a rich, flexible, modern development environment forcreating more secure database applications.
4. Share data across multiple platforms, applications, and devices to make it easier to connect internal and external systems.
5. Deliver robust, integrated business intelligence solutions that help drive informed business decisions and increase productivity across your entire organization.
6. Control costs without sacrificing performance, availability, or scalability.

To learn more about the advancements SQL Server 2008 willdeliver in three key areas: enterprise data management, developer productivity, and business intelligence. In today's connected world, data and the systems that manage that data must always be available to your users. With SQL Server 2008, users and IT professionals across your organization will benefit from reduced application downtime, increased scalability and performance, and tight security controls.

SQL Server 2008 will include enhancements to enterprise data management in the following areas:

- Availability:

Investments in high availability technologies, additional backup and restore capabilities, and replication enhancements will enable enterprises to build and deployhighly reliable applications.

- Scalability:

Scalability advancements such as partitioning, snapshot isolation, and 64-bit support will enable you to build and deploy your most demanding applications using SQL Server 2008.

- Security:

Enhancements such as “secure by default” settings and an enhanced security model will help provide a high level of security for your enterprise data.

- Manageability:

A new management tool suite, expanded\self-tuning capabilities, and apowerful new programming model will increase the productivity of database administrators.

1. XML and Web services:

SQLServer 2008 will support both relational and XML data natively, so enterprises can store, manage, and analyze data in the format that best suits their needs. Support for existing and emerging open standards such as Hypertext Transfer Protocol (HTTP), XML, Simple Object Access Protocol(SOAP), XQuery, and XML Schema Definition (XSD) will also facilitate communication across extended enterprise systems.

Business Intelligence

The challenge and promise of business intelligence revolves around providing employees with the right information, at the right time. Accomplishing this vision demands a business intelligence solution that is comprehensive, secure, integrated with operational systems, and available all day, every day. SQL Server will help companies to achieve this goal with SQL Server 2008. Business intelligence advancements will include:

1. Integrated platform:

   SQL Server 2008 will deliver an end-to-end business intelligence platform with integrated analytics including online analytical processing (OLAP); data mining; extract, transformation, and load (ETL) tools; data warehousing; and reporting functionality. This document is developed prior to the product’s release to manufacturing, and as such, we cannot guarantee that all details included herein will be exactly as what is found in the shipping product. The information contained in this document represents the current view of Microsoft Corporation on the issues discussed as of the date of publication. Because Microsoft must respond to changing market conditions, it should not be interpreted to be a commitment on the part of Microsoft, and

Microsoft cannot guarantee the accuracy of any information presented after the date of publication. The information represents the product at the time this document was printed and should be used for planning purposes only. Information subject to change at any time without prior notice. This document is for informational purposes only.

2. Improved decision making:

   Advancements to existing business intelligence features, such as OLAP and data mining, and the introduction of a new reporting server will provideenterprises with the ability to transform information into better business decisions at all organizational levels.

3. Security and availability:

   Scalability, availability, and security enhancements will help to provide users with uninterrupted access to business intelligence applications and reports.

4. Enterprise-wide analytical capabilities:

   An improved ETL tool will enable organizations to more easily integrate and analyze data from multiple heterogeneous information sources. By analyzing data across a wide array of operational systems, organizations may gain a competitive edge through a holistic understanding of their business.

5. Interoperability:

   Through deep support for industry standards, Web services, and the Microsoft .NET Framework, SQL Server 2008 will support interoperability with multiple platforms, applications, and devices.

   DeveloperProductivity One of the key barriers to developer productivity has been the lack of integrated tools for database development and debugging. SQL Server 2008 will provide advancements that fundamentally change the way that database applications are developed and deployed. Enhancements for developer productivity will include:

   1. Improved tools:

      Developers will be able to utilize one development tool for Transact-SQL, XML, Multidimensional Expression (MDX), and XML for Analysis (XML/A).Integration with the Visual Studio® development environment will provide more efficient development and debugging of line-of-business and business intelligence applications.

   2. Expanded language support:

      With the common language runtime (CLR) hosted in the database engine, developers will be able to choose from a variety of familiar languages to develop database applications, including Transact-SQL, Microsoft Visual Basic .NET, and Microsoft Visual C# .NET.

CHAPTER 3
SYSTEM DESIGN

3.1 INTRODUCTION

Design is the second phase in the system development life cycle. Software design is the first of the three technical activities in the software development process such as design, code writing and testing. During this phase the analyst schedules design activities works with the user to determine the various data inputs to the system, plans how data flow through the system, design required outputs and write program specifications. Again, the analyst's activities on focus on solving a user's problem in logical terms. During this second step analysts employ a variety of tools such as data flow diagram, entity relationship, data dictionaries and grant chart. The system design converts the theoretical solutions introduced by the feasibility study into a logical reality.

3.2.INPUT DESIGN

input design is the process of converting user-oriented input into a computer- based format. The data flow diagram indicates logical data flows, data store source and destinations. Input data are collected and organized into a group of numbered data. Once identified appropriate input data media are selected for processing. The goal of input design is to make data entry as easy and free from errors as much as possible. Proper data validation checking is exercised to correct the mistakes made during data entry.

3.3.OUTPUT DESIGN

Output design is an ongoing activity almost from the beginning of the project. Computer output is the most important and direct source of information to the user. Output design is a process that involves designing necessary outputs that must be given to various user according to their requirements. Efficient output design should improve the system's relationship with the user and help in decision making. The system has got the capability to display standard screen layouts. These layouts should be designed around the output requirements and they must be designed with utmost care and details in the screen layouts must be simple, descriptive and clear to the user. While designing a systems output, the analyst must make several independent decisions. Every system produces reports. No matter what the content of the report is. The following guidelines can be applied to any report.

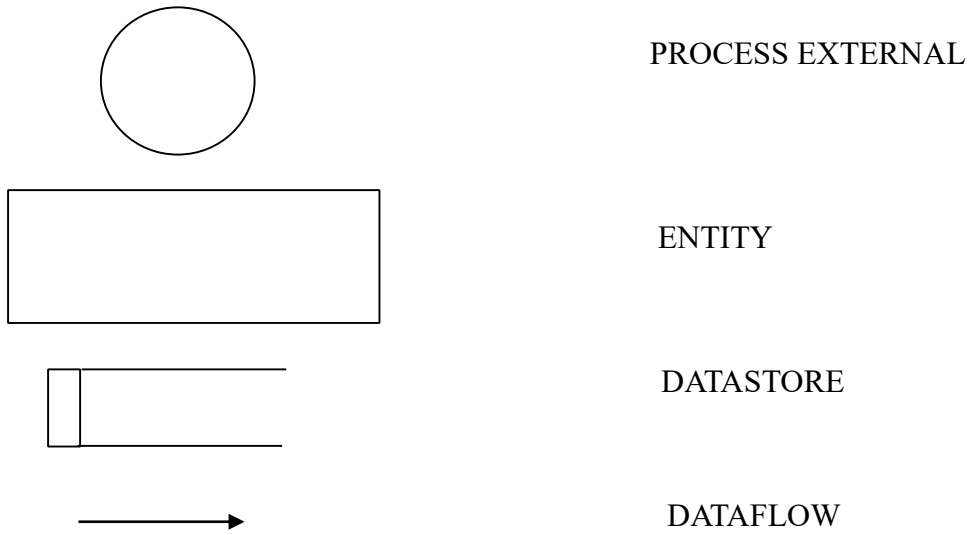
- The information should be clear, accurate, concise and restricted to relevant data
- Reports should have titles and descriptive headings for columns of data, numbered pages and so on.
- The reports content should be in a logical arrangement so that user can easily locate what they need.

3.4. DATA FLOW DIAGRAMS (DFD)

Data flow diagrams were proposed by Larry Constantine. It is a graphical representation of the flow of data through an information system, modeling its process aspects. A DFD is often used as a preliminary step to create an overview of the system, which can later be elaborated. DFDs can also be used for the visualization of data processing.

A DFD shows what kind of information will be input to and output from the system, where the data will come from and go to, and where the data will be stored. It does not show information about the timing of process or information about whether processes will operate in sequence or in parallel.

DATA FLOW NOTATIONS



1.Process/Function

A process represents activities in which data is manipulated by being stored or retrieved or transferred in some way. The process transforms the input data into output data.

2.External Entities

This represents any outside agency, which interact with the system. It represents the source or destination of data for the system under the consideration.

3.**Data Stores/Database**

A data store represents a logical life. Data store is represented by open ended rectangle.

Each data store is connected to a process by means of a data flow symbol.

4. **Data Flows**

The data flow represents in an interface among different components in a DFD. It represents flow of data between two processes or between a process and an external entity or between a process and data store.

A few guidelines to aid the derivation of a DFD :

- The level 0 DFD should depict the software/system as a single bubble.
- Primary input and output should be noted.
- Refinement should begin by isolating candidate processes, data objects and the stores to be represented in the next level.
- All arrows and bubbles should be labeled with a meaningful name.
- Information flow continuity must be maintained from level to level.

One bubble at a time should be refined.

The level 0 DFD is expanded to level 1 DFD. The process represented in the level 1 DFD can be further refined to level 2 DFD. Information flow continuity is maintained between levels. The refinement continues until each bubble performs a simple function. That is, until the process represented by the bubble performs a function that would be easily implemented as a program.

The DFD serves two purposes :

- To provide an indication of how data is transformed as they move through the system, and to depict the functions (and sub functions) that transforms the data flow.
- DFD provide additional information that is used during the analysis of the information domain and serves as a basis for the modeling of functions.

LEVEL ZERO / Context Diagram

Fig 3.4.1

LEVEL 1

Fig 3.4.2

LEVEL 2 User

Fig 3.4.3

LEVEL 3 Member

Fig 3.4.3

LEVEL 4 Co-ordinator

Fig 3.4.4

LEVEL 5 Convenor

Fig 3.4.5

3.5. STRUCTURE CHART

A structure chart is a chart which shows the breakdown of a system to its lowest manageable levels. They are used in structured programming to arrange program modules into a tree. Each module is represented by a box, which contains the module's name. The tree structure visualizes the relationships between modules.

A structure chart is a top-down modular design tool, constructed of squares representing the different modules in the system, and lines that connect them. The lines represent the connection between activities and sub activities as they are used in organization charts.

Fig 3.5

3.6 DATA BASE DESIGN

A relational database is a collection of data items organized as a set of formally-described tables from which data can be accessed or reassembled in many different ways without having to reorganize the database tables. The relational database was invented by E. F. Codd at IBM in 1970.

A database is an organized collection of data. The data is typically organized to model aspects of reality in a way that supports processes requiring information. It refers to a set of related data and the way it is structured or organized.

Database design is the process of producing a detailed data model of a database. The process of doing database design generally consists of a number of steps which will be carried out by the database designer. Usually, the designer must:

- Determine the relationships between the different data elements.
- Superimpose a logical structure upon the data on the basis of these relationships.

Normalization

Normalization is a design technique that is widely used as a guide in designing relational databases. It is the process of decomposing the attributes in an application, which results in a set of tables with very simple structure. There are two goals of the normalization process:

- Eliminate redundant data
- Ensure data dependencies

The database community has developed a series of guidelines for ensuring that databases are normalized. These are referred to as normal forms. It is referred as 1NF, 2NF, and 3NF along with BCNF.

Constraints in creating a table:

- **Candidate Key :** A candidate key is a combination of one or more columns, the value of which uniquely identifies each row of a table.
- **primary Key :** The primary key of a table is one of the candidate keys that you give some characteristics. We can have only one primary key, and a primary key column cannot contain NULLs.
- **Foreign Key :** A foreign key is a combination of columns with values based on primary key values from another table. A foreign key constraint, also known as

Referential Integrity constraint, specifies that the values of the foreign key correspond to actual values of the primary key in the other table.

- **CHECK Constraint :** Many columns must have values that are within a certain range or that satisfy certain conditions. With a CHECK constraint, we can specify an expression that must always be true for every row in the table.

Rules of Data Normalization

- **1 NF :**
  1 NF specifies eliminate repeating groups. It make a separate table for each set of related attributes, and give each table a primary key.
- **2 NF :**
  2 NF specifies eliminate redundant data. If an attribute depends on only one part of a multi-valued key, remove it to a separate table.
- **3 NF :**
  3NF specifies eliminate columns not dependent on key. If attributes do not contribute to a description of the key, remove them to a separate table.
- **BCNF :**
  It means Boyce-Codd Normal Form. If there are non-trivial dependencies between candidate key attributes, separate them out into distinct tables.

Table 3.6.1 tbl User Table

Field Name	Django Field Type	Data Type	Size	Constraint	Notes
ID	BigAutoField	integer	9 quintillion+	Primary Key,Auto Register	User identification number
Username	CharField	Varchar	150	NOT NULL,Unique	Login using username
Password	CharField	Varchar	128	NOT NULL	Stored hashed type,not plain.
first_name	CharField	Varchar	150	Optional	Required for updation.
email	EmailField	Varchar	254	Optional	Not unique

Table 3.6.2 tbl User Profile Table

Field Name	Django Field Type	Data Type	Size	Constraint	Notes
user	OneToOneField (User)	-	-	User	Links one record to exactly one other record (unique relationship).
phone_number	CharField	Int	10	Optional	User's phone number
role	CharField	Varchar	20	Choice (Automatic)	Choose roles based on purpose
gender	CharField	Varchar	10	Choice	To determine which gender is user
address	TextField	Text	No limit	Optional	To determine where the user lives.
photo	ImageField	Varchar	100 (path characters)	Optional	User's photo for identify.
created_at	DateTimeField	datetime	-	Auto Create	To know when the user is created.

Table 3.6.3 tbl Coordinate Registration Table

Field Name	Django Field Type	Data Type	Size	Constraint	Notes
user	OneToOneField (User)	-	-	User, Auto create	Links one record to exactly one other record (unique relationship).
adno	CharField	Int	255	Required	Admission number
department	CharField	Varchar	50	Required, Choice	Select coordinator's department.
batch	CharField	Varchar	10	Required	Coordinators batch year.
current_year	CharField	Varchar	15	Required	Current year that the coordinator studying.

Table 3.6.4 tbl MEMBER REGISTRATION TABLE

Field Name	Django Field Type	Data Type	Size	Constraint	Notes
user	OneToOneField (User)	-	-	User, Auto create	Links one record to exactly one other record (unique relationship).
adno	CharField	Int	255	Required	Admission number
department	CharField	Varchar	50	Required, Choice	Select user's department.
batch	CharField	Varchar	10	Required	Students batch year.
current_year	CharField	Varchar	15	Required	Current year that the student studying.
duty	CharField	Varchar	50	Choice	To define the member duty.

Table 3.6.5 tbl PENDING MEMBER ADD REQUEST TABLE

Field Name	Django Field Type	Data Type	Size	Constraint	Notes
user	OneToOneField (User)	-	-	User, Auto create	Links one record to exactly one other record (unique relationship).
isApproved	BooleanField	Boolean	-	Default=False	Is the member approved or not
isPending	BooleanField	Boolean	-	Default=False	Is the member in pending or not.

reason	CharField	Varchar	255	Required for rejection	The reason for request getting reject

Table 3.6.6 tbl NOTIFY MODEL

Field Name	Django Field Type	Data Type	Size	Constraint	Notes
user	ForeignKey (User)	-	-	User, Auto create	Links many records to one record (many-to-one relationship).
title	CharField	Varchar	255	Required	Title of notification
description	TextField	Text	-	Required	Description of notification
is_completed	BooleanField	Boolean	-	Default=False	Is this notification completed
program_date	DateField	date	-	Required	When the program
announced_time	DateTimeField	datetime	-	Auto Create	When this notification is published

Table 3.6.7 tbl FINANCE TABLE

Field Name	Django Field Type	Data Type	Size	Constraint	Notes
user	ForeignKey (User)	-	-	User, Auto create	Links many records to one record (many-to-one relationship).
collection_type	CharField	Varchar	18	Choice,Required	Which is the collection type
description	TextField	Text	-	Required	Description of collection
collection_date	DateField	date	-	Required	When will collection initiate
announced_date	DateField	date	-	Auto Create	When this collection is published
image	ImageField	Varchar	100(Char)	Required	Photo of collection record

Table 3.6.8 tbl KIT RECEIVER TABLE

Field Name	Django Field Type	Data Type	Size	Constraint	Notes
user	ForeignKey (User)	-	-	User, Auto create	Links many records to one record (many-to-one relationship).
name	CharField	Varchar	30	Required	Name of kit receiver
age	IntegerField	Int	-	Required	Age of people
family	TextField	Text	-	Required	Family address of people
address	TextField	Text	-	Required	Address of people
location	URLField	Varchar	255	Required	Location of people
photo	ImageField	Varchar	200(Chars)	Required	Photo of people
Announced_date	DateField	date	-	Auto create	When this record is published

3.7 ER-Diagram

Database designs also include ER (Entity-relationship model) diagrams. An ER diagram is a diagram that helps to design databases in an efficient way. Attributes in ER diagrams are usually modeled as an oval with the name of the attribute, linked to the entity or relationship that contains the attribute. Within the relational model the final step can generally be broken down into two further steps, that of determining the grouping of information within the system, generally determining what are the basic objects about which information is being stored, and then determining the relationships between these groups of information, or objects. This step is not necessary with an Object database

The E-R Model: The enterprise is viewed as set of

- Entities
- Relationships among entities Symbols used in E-R Diagram

- Entity – rectangle
- Attribute – oval
- Relationship – diamond
- Link – line

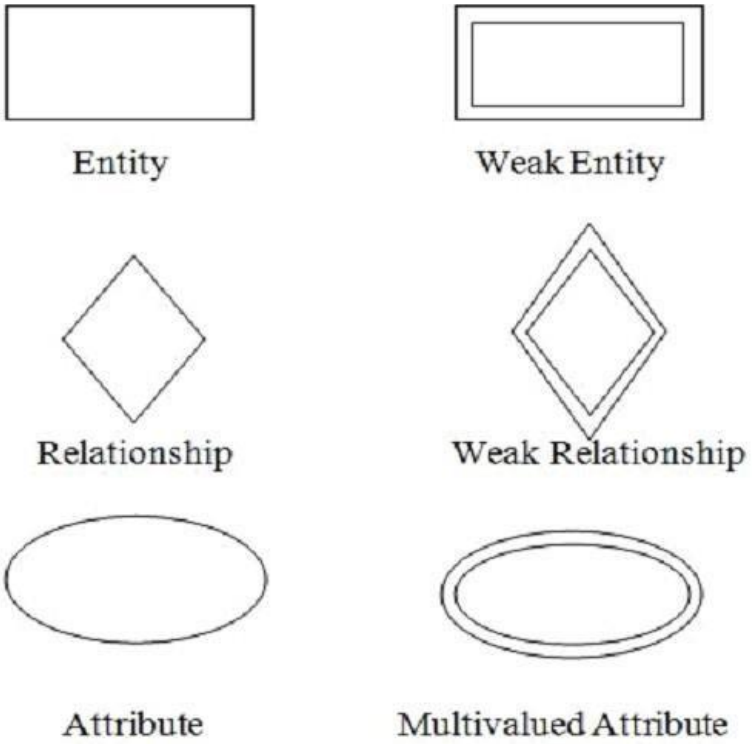


Fig: Symbols of ER diagram

Entity

An entity can be a person, place, event, or object that is relevant to a given system. For example, a school system may include students, teachers, major courses, subjects, fees, and other items. Entities are represented in ER diagrams by a rectangle and named using singular nouns.

Weak Entity

A weak entity is an entity that depends on the existence of another entity. In more technical terms it can be defined as an entity that cannot be identified by its own attributes. It uses a foreign key combined with its attributes to form the primary key. An entity like order item is a good example for this. The order item will be meaningless without an order so it depends on the existence of the order.

Attribute

An attribute is a property, trait, or characteristic of an entity, relationship, or another attribute. For example, the attribute Inventory Item Name is an attribute of the entity Inventory Item. An entity can have as many attributes as necessary. Meanwhile, attributes can also have their own specific attributes. For example, the attribute "customer address" can have the attributes number, street, city, and state. These are called composite attributes. Note that some top level ER diagrams do not show attributes for the sake of simplicity. In those that do, however, attributes are represented by oval shapes.

Relationship

A relationship describes how entities interact. For example, the entity "carpenter" may be related to the entity "table" by the relationship "builds" or "makes". Relationships are represented by diamond shapes and are labeled using verbs.

Multivalued Attribute

If an attribute can have more than one value it is called a multivalued attribute. It is important to note that this is different to an attribute having its own attributes. For example a teacher entity can have multiple subject values. .

Derived Attribute

An attribute based on another attribute. This is found rarely in ER diagrams. For example, for a circle the area can be derived from the radius

Fig 3.7

3.8 USE CASE DIAGRAM

To model a system the most important aspect is to capture the dynamic behavior. To clarify a bit in details, dynamic behavior means the behavior of the system when it is running or operating. So only static behavior is not sufficient to model a system rather dynamic behavior is more important than static behavior. In UML there are five diagrams available to model dynamic nature and use case diagram is one of them.These internal and external agents are known as actors. So use case diagrams are consists of actors, use cases and their relationships. The diagram is used to model the system/subsystem of an application. A single use case diagram captures a particular functionality of a system.The purposes of use case diagrams can be as follows:

- Used to gather requirements of a system.
- Used to get an outside view of a system.
- Identify external and internal factors influencing the system.

The main purpose of a use case diagram is to show what system functions are performed for which actors. Roles of the actors in the system can be depicted. Use cases are used in almost every project. They are helpful in exposing requirements and planning the project.

The two main components of a use case diagram are use cases and actors.

USE CASE: ADMIN